Assignment 4: Private information retrieval

Muhammad Ayain Fida Rana
Department of Computer Science and Technology
University of Cambridge
15 JJ Thomson Avenue, Cambridge CB3 0FD, United Kingdom
mafr2@cam.ac.uk

1 Introduction

In this assignment, I implemented a PIR scheme using symmetric LWE encryption from scratch. My implementation closely follows Henzinger et al. (2023) and the lecture notes for the protocol specifications, with the message formats and encoding schemes defined by me. The code is publicly available at https://github.com/ayainfida/p79-assignment4-pir.

2 Implementation Architecture

The core implementation is split across modular components, briefly introduced here, with key design choices discussed in Section 3.

Ring Elements (`ring.py`). Defines an integer modulo q ring, `ModInt`, supporting arithmetic and bitwise operations.

Database (`db.py`). Implements a `Database` class that supports `get` and `set` operations.

LWE (`lwe.py`). Implements the core LWE methods for sampling the error e and secret vector s, and generating the matrix A.

Encoding/Decoding (`encoding.py`). Handles serialization for the `PIRMessage` payloads.

PIR (`pir.py`). Exposes the PIR interface through the `PIRServer` and `PIRClient` classes, each implementing the role-specific operations.

Defaults (`defaults.py`). Defines the modulus q of the ring, the database size N, and the secret vector dimension n.

3 Implementation Design Choices

3.1 Ring Representation

Similar to the field representation in Assignment 2, I introduced an immutable `ModInt` base class, with `RingElement` (modulo $q = 2^{32}$) inheriting from it. The `ModInt` class ensures that every value is reduced modulo q during initialization.

I intentionally allowed arithmetic operations between `RingElement` ($\mathbb{Z}_q$) objects and integers ($\mathbb{Z}$) to simplify the implementation of LWE operations, which involve matrix multiplication; otherwise, many explicit conversions would be required, making the code unnecessarily verbose.

3.2 Database

I implemented a basic Database (DB) that stores data in the form of a `NumPy`[1] array. The DB has an effective size[2] of N, with the layout dependent on the PIR scheme: a 1D array of dimension N in the case of the `NAIVE` scheme, and a 2D array of dimensions $(\sqrt{N}, \sqrt{N})$ for the `SQRT` and `OPTIMIZED_SQRT` schemes. The DB supports simple `get` and `set` operations that abstract the scheme-dependent index mapping.

3.2.1 Database updates

In the `OPTIMIZED_SQRT` scheme, the DB audits all updates, which can then be used by `PIRServer` to update A' incrementally rather than recomputing it. In particular, a single DB update only affects one row of A'. Since each row of A has length n, the update can be performed in $O(n)$ rather than $O(\sqrt{N}\,n)$ in the case of full recomputation of the corresponding row in $A' = DA$.

```
# Extract bit representation of old and new value
old_bits = np.array([(old >> b) & 1 for b in range(n_bits)], dtype=int)
new_bits = np.array([(new >> b) & 1 for b in range(n_bits)], dtype=int)

# Calculate the delta
delta_bits = new_bits - old_bits

# Update the affected row of A'
# A[:, col, :] has length n  -> update cost O(n * n_bits)
self.A_prime[:, row, :] += delta_bits[:, None] * A[:, col, :]
```

3.2.2 Multi-bit representation

I chose to support retrieval of both `bool`[3] and `uint8` data types. This is why the dimensions become a bit unclean, i.e. a 3D array, but a more convenient way to understand this is as regular DBs (1D or 2D) stacked[4] on top of each other. With `uint8` having 8 bits, it can be thought of as 8 single-bit databases stacked together, as illustrated in Figure 1. In this way, the dimensions of the DB (D) become either (n_{bits}, N) or $(n_{\text{bits}}, \sqrt{N}, \sqrt{N})$.

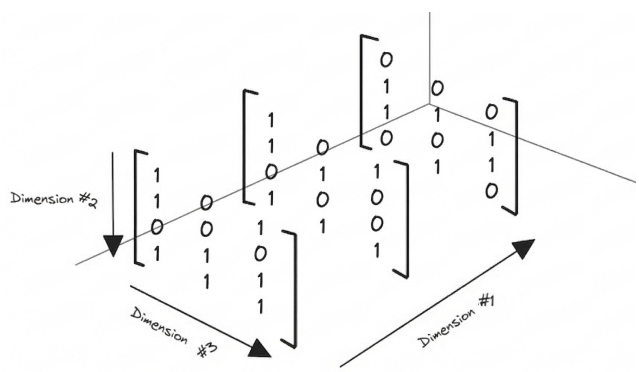

Figure 1: Multi-bit DB representation, where each slice corresponds to one bit position of the stored values.

[1] allows for efficient vectorized operations
[2] For simplicity, N is assumed to be a square number
[3] As `bool` has only 2 states, it mimics single-bit retrieval
[4] a more appropriate way is to think of them as tensors

3.3 Symmetric LWE Encryption

Since the PIR scheme is based on symmetric LWE encryption, I implemented helper methods to generate the following parameters:

- Public matrix[5] $A \in \mathbb{Z}_q^{\text{n_bits} \times M \times n}$ generated uniformly over $\mathbb{Z}_q$ using a random seed, which serves as a hint in the `OPTIMIZED_SQRT` scheme.
- Secret key $s \in \mathbb{Z}_q^{\text{n_bits} \times n}$ sampled uniformly from $\mathbb{Z}_q$.
- Error vector $e \in \mathbb{Z}_q^{\text{n_bits} \times M}$ sampled from a bounded interval $[-B, B]$.

The dimensions are stated in a bit-sliced form to generalize the retrieval of multi-bit values. At its core, this corresponds to running `n_bits` independent LWE instances in parallel, one for each bit position. Moreover, the query vector q can be thought of as a one-hot encoded vector replicated `n_bits` times. The final value can then be reconstructed by concatenating the recovered bits.

Default Parameters. I chose the modulus $q = 2^{32}$ so that arithmetic can make use of the full 32-bit range before wrapping around. Similarly, the size of DB N is set to 2^{10}, so it has at least 1000 values. The parameter n describes the dimension of the secret vector; a higher value provides greater security at the cost of runtime. To keep the test cases reasonably fast, I chose $n = 64$. Lastly, the parameter B, which bounds the error vector, is set to 3, satisfying the condition $B \ll q$.

3.4 PIR Scheme

3.4.1 Message Format Design

Every message exchanged during communication is of type `PIRMessage` and is formatted as:

$$\underbrace{\texttt{PIRMessageType}}_{1\text{ B}} \parallel \underbrace{\texttt{PIRScheme}}_{1\text{ B}} \parallel \underbrace{\texttt{payload}}_{\texttt{payload_len B}}$$

The payloads of each message type are defined below:

$$\text{HINT} \quad \underbrace{\texttt{seed}}_{32\text{ B}} \parallel \underbrace{\texttt{A_prime}}_{\text{n_bits} \times M \times n \times 32\text{ B}}$$

$$\text{QUERY/ANSWER} \quad \underbrace{\texttt{A}}_{\text{n_bits} \times M \times n \times 32\text{ B}} \parallel \underbrace{\texttt{c}}_{\text{n_bits} \times M \times 32\text{ B}} \quad \textbf{OR} \quad \underbrace{\texttt{c}}_{\text{n_bits} \times M \times 32\text{ B}}$$

`PIRMessage` can be thought of as a record wrapper that wraps each protocol message with a header[6] before the payload is decoded.

3.4.2 Interaction flow

To make logic of the PIR scheme clearer, I created two classes, `PIRServer` and `PIRClient` that inherit from the base `PIR` class. This allowed me to separate the processing of messages for the client and the server while maintaining a common handler interface.

```
def handle_message(self, message: bytes) -> bytes | int | None:
    pir_message = PIRMessage.from_bytes(message)

    if pir_message.msg_type == PIRMessageType.QUERY:
        return self.answer(pir_message.payload)
    elif pir_message.msg_type == PIRMessageType.HINT:
```

[5] Let M denote the DB dimension according to the scheme (N or $\sqrt{N}$)

[6] `PIRMessageType` and `PIRScheme`

```
        self.download_hint(pir_message.payload)
    elif pir_message.msg_type == PIRMessageType.ANSWER:
        return self.recover(pir_message.payload)
    else:
       raise ValueError(f"Invalid message type. Provided type: {type(message)}")
```

Figure 2 illustrates the messages exchanged when a client wants to retrieve the (i, j)-th entry from the DB for non-optimized schemes.

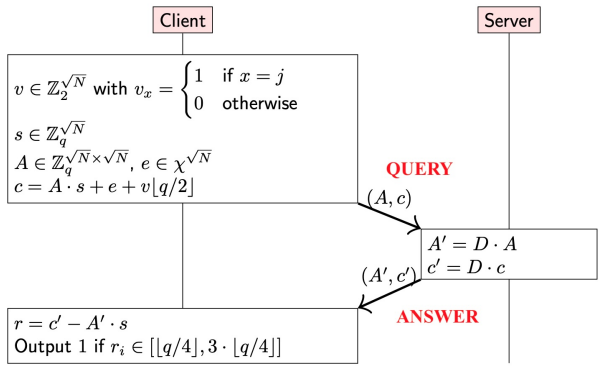

Figure 2: PIR message flow between client and server for `SQRT` scheme.

3.4.3 *Optimized PIR scheme*

For the `OPTIMIZED_SQRT` scheme, the server's `setup()` method also generates a hint. The server includes the seed[7] and the matrix A' in the payload. The client then downloads this hint using the `download_hint(.)` method. In this way, subsequent communication only requires the corresponding c to be transmitted.

In the case of database updates (Section 3.2.1), the server invokes the `update()` method, which generates a new hint for the client to download.

3.4.4 *Multi-bit query processing*

Given my implementation supports retrieval of `uint8`[8] data type (Section 3.2.2), the computation of $As + e + \lfloor \frac{q}{2} \rfloor \times v$ is no longer trivial, because `NumPy` does not directly support matrix multiplication between $A \in \mathbb{Z}_q^{\text{n_bits} \times M \times n}$ and $s \in \mathbb{Z}_q^{\text{n_bits} \times n}$.

To address this, I add a singleton dimension to s, transforming it to $\mathbb{Z}_q^{\text{n_bits} \times n \times 1}$, which allows `NumPy` to perform batched matrix multiplication across the bit dimension.

```
(self.A @ self.s[..., None]).squeeze(-1)
```

The minimal communication guarantees are still met because batching only increases the payload size by `n_bits`, without changing the overall complexity.

4 Testing and Validation

Individual tests are documented in `tests/*.py`; here, I provide a high-level overview of the testing strategy.

[7] used to generate A

[8] My implementation works for larger integers as well

PIR API Testing. Validated agreement of the retrieved value across all schemes and supported data types. Tested retrieval for both valid and invalid queries.

LWE. Validated that generated parameters lie within the expected bounds, and verified the encryption–decryption round-trip and homomorphic properties.

DB Testing. Validated the correctness of `get` and `set` operations across both naive and square-root based layouts.

Encoding. Tested the round-trip property for encoding/decoding of `PIRMessage` payloads, as well as length and type validation.

Ring Operations. Tested the correctness of the defined arithmetic and bitwise operations.

5 Discussion

5.1 Limitations and Production Considerations

Starting with the DB, `NumPy` stores both `bool` and `uint8` values as 1 byte, so having two separate types did not help with the storage efficiency. Moreover, the assumption that DB size N is a square number can be relaxed by applying padding internally to support the square layout.

For the error vector, I used bounded noise instead of the discrete Gaussian distribution used by Henzinger et al. (2023). I believe production standards would need to use Gaussian noise to align with standard LWE hardness assumptions.

My parameter choices are mostly meant to keep the test suite fast. In practice, the dimension n would need to be much larger to achieve meaningful security guarantees.

5.2 Observations and Uncertainties

Initially, I considered implementing multi-bit retrieval by querying the database bit by bit in a loop. However, I chose to perform batched operations, which introduces an interesting trade-off between the number of two-way exchanges and the payload size per exchange.

Although I optimized the computation of A' on the server in the case of a DB update, the complete A' matrix is still sent to the client, which is expensive for large a DB. This could be improved by sending only a log of deltas[9] for A'. Another optimization would be to cache the server's response c' for a query (i, j). Since c' corresponds to the j-th column of D, if another query (i', j) is issued, the client could reuse[10] the cached response without contacting the server again.

Moreover, I tested the boundary conditions for the error vector by increasing B to q. Retrieval no longer worked beyond 2^{28}, suggesting that the noise magnitude must remain well below the threshold $q/4$.

References

Alexandra Henzinger, Matthew M Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. One server for the price of two: Simple and fast {Single-Server} private information retrieval. In *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 3889–3905, 2023.

[9] of the form (delta, index)

[10] *I am ignoring the staleness concerns for now*